

■ OOP in Python

Python — Object-Oriented Programming: Classes, Objects, Inheritance & More

#	Topic
1	Class
2	Object
3	__init__ & Dunder Methods
4	Parameters vs Arguments
5	Class Variable vs Instance Variable
6	@classmethod
7	@staticmethod
8	Encapsulation & Access Modifiers
9	Getters & Setters
10	Inheritance
11	Method Resolution Order (MRO)
12	Abstraction
13	Polymorphism
14	Duck Typing

1 • Class

A **class** is a **blueprint** for creating objects. It defines the attributes (data) and methods (behaviour) that every object created from it will have. Think of it as a cookie-cutter — the class is the cutter, each cookie is an object.

Python

```
1 class Dog:
2     species = "Canis familiaris" # class variable
3     def __init__(self, name, age):
4         self.name = name # instance variable
5         self.age = age
```

2 • Object

An **object** is a concrete **instance** of a class. Each object has its own copy of instance variables but shares class variables with all other instances.

Python

```
1 buddy = Dog("Buddy", 3) # creating an object (instance)
2 print(buddy.name) # Buddy
3 print(buddy.species) # Canis familiaris
```

3 • `__init__` & Dunder Methods

The `__init__` method is the **constructor** — Python calls it automatically every time a new object is created. It sets up the initial state of the object.

Dunder (double-underscore) methods are **special/magic methods** that Python calls implicitly. Common dunder methods:

Dunder Method	When Python Calls It
<code>__init__(self, ...)</code>	Object is created — sets up initial state
<code>__str__(self)</code>	<code>str(obj)</code> or <code>print(obj)</code> — human-readable string
<code>__repr__(self)</code>	<code>repr(obj)</code> — unambiguous developer representation
<code>__len__(self)</code>	<code>len(obj)</code>
<code>__add__(self, other)</code>	<code>obj1 + obj2</code>

Python

```
1 class Dog:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     def __str__(self):
7         return f"{self.name} is {self.age} years old"
8
9     buddy = Dog("Buddy", 3)
1    print(buddy) # Buddy is 3 years old
0
```

4 • Parameters vs Arguments

Term	Meaning
Parameter	Placeholder variable in the function/method definition
Argument	Actual value passed when calling the function/method

Python

```
1 def greet(name): # "name" is a PARAMETER
2     print(f"Hello {name}")
3
4     greet("Rohi") # "Rohi" is an ARGUMENT
```

5 • Class Variable vs Instance Variable

Variable Type	Where Declared	Shared?
Class variable	Inside the class, outside all methods	Yes — shared by all instances
Instance variable	Inside a method (usually <code>__init__</code>)	No — unique to each object

Python

```
1 class Circle:
2     pi = 3.14159 # class variable – same for all circles
3
4     def __init__(self, radius):
5         self.radius = radius # instance variable – unique per object
6
7     c1 = Circle(5)
8     c2 = Circle(10)
9     print(Circle.pi) # 3.14159
1    print(c1.radius, c2.radius) # 5 10
0
```

6 • @classmethod

A **class method** receives the **class itself** (cls) as its first argument instead of the instance (self). It is defined with the **@classmethod** decorator. Useful for alternative constructors or factory methods.

Python

```
1 class Dog:
2     def __init__(self, name, age):
3         self.name = name
4         self.age = age
5
6     @classmethod
7     def from_birth_year(cls, name, birth_year):
8         return cls(name, 2024 - birth_year) # cls = Dog
9
1    buddy = Dog.from_birth_year("Buddy", 2021)
2
3    print(buddy.age) # 3
4
```

■ Use `@classmethod` when you need to access or modify the class itself, not a specific instance. A common pattern is an alternative constructor.

7 • @staticmethod

A **static method** receives **neither** `self` nor `cls`. It behaves like a regular function but lives inside the class for organisational reasons. Defined with the `@staticmethod` decorator.

Python

```
1 class MathUtils:
2     @staticmethod
3     def add(a, b):
4         return a + b
5
6    print(MathUtils.add(3, 4)) # 7 (no instance needed)
```

Decorator	First Argument	Use Case
(none)	self	Regular instance method — accesses instance data
@classmethod	cls	Needs access to the class — alternative constructors
@staticmethod	(none)	Utility function logically grouped with the class

8 • Encapsulation & Access Modifiers

Encapsulation means bundling data (variables) and behaviour (methods) together inside a class, and controlling what is visible to the outside world. Python uses naming conventions to signal access levels:

Access Modifier	Python Syntax	Accessible From
Public	name	Anywhere — inside and outside the class
Protected	<code>_name</code>	Class itself and its subclasses only
Private	<code>__name</code>	Only within the class (name-mangled)

Python

```
1 class BankAccount:
2     def __init__(self, owner, balance):
3         self.owner = owner # public
4         self._bank = "MyBank" # protected
5         self.__balance = balance # private (name-mangled to _BankAccount__balance)
6
7     def deposit(self, amount):
8         self.__balance += amount
9
10    def get_balance(self): # getter for private variable
11
12    return self.__balance
13
```

■ Python does not enforce access modifiers at runtime — they are conventions. A leading double underscore triggers name mangling (`self.__x` becomes `_ClassName__x`), making accidental access harder but not impossible.

9 • Getters & Setters

Because private variables cannot be accessed directly from outside the class, we expose them through **getter** (read) and **setter** (write) methods. Python provides the **@property** decorator as a clean, Pythonic way to do this.

Python

```
1 class Temperature:
2     def __init__(self, celsius):
3         self.__celsius = celsius
4
5     @property
6     def celsius(self): # getter
7         return self.__celsius
8
9     @celsius.setter
10    def celsius(self, value): # setter with validation
11
12        if value < -273.15:
13
14            raise ValueError("Temperature below absolute zero!")
15
16        self.__celsius = value
17
18
19    t = Temperature(25)
20
21    print(t.celsius) # 25 (calls getter)
22
23    t.celsius = 30 # calls setter
24
```

■ Prefer `@property` over plain `get_x()` / `set_x()` methods — it keeps the interface clean (`t.celsius` instead of `t.get_celsius()`) while still allowing validation logic.

10 · Inheritance

Inheritance lets a **child class** acquire the attributes and methods of a **parent class**, promoting code reuse. The child can also override or extend any inherited behaviour.

Python

```
1 class Animal: # Parent class
2     def __init__(self, name):
3         self.name = name
4
5     def speak(self):
6         return f"{self.name} makes a sound"
7
8 class Dog(Animal): # Child class
9     def speak(self): # override parent method
1        return f"{self.name} barks"
0
1
1
1 class Cat(Animal):
2
3     def speak(self):
4
5         return f"{self.name} meows"
6
7
1 d = Dog("Buddy")
6
1 print(d.speak()) # Buddy barks
7
```

super() — calling the parent's method from the child:

Python

```
1 class Dog(Animal):
2     def __init__(self, name, breed):
3         super().__init__(name) # call Animal.__init__
4         self.breed = breed
```

11 · Method Resolution Order (MRO)

When Python looks up a method or attribute, it follows the **MRO** — a left-to-right, depth-first search order defined by the C3 linearisation algorithm.

Search order: **Current class** → **Super class** → **Super-super class** → ... → **object**

Python

```
1 class A:
2     def hello(self): return "A"
3
4 class B(A):
5     def hello(self): return "B"
6
7 class C(A):
8     pass
9
10 class D(B, C): # multiple inheritance
11
12     pass
13
14 d = D()
15
16 print(d.hello()) # B (follows MRO: D -> B -> C -> A)
17
18 print(D.__mro__) # (, , , , )
19
```

■ Always check `ClassName.__mro__` when debugging multiple inheritance — it shows the exact order Python will search for any attribute or method.

12 · Abstraction

Abstraction means hiding implementation details and exposing only what is necessary. In Python, abstract classes are created using the **abc** module. An **abstract method** is declared in the parent but **must be implemented** in every child class — the parent compulsorily forces the child to define it.

Python

```
1  from abc import ABC, abstractmethod
2
3  class Shape(ABC): # Abstract base class
4      @abstractmethod
5      def area(self): # every subclass MUST implement this
6          pass
7
8      @abstractmethod
9      def perimeter(self):
1         pass
10
11
12     class Rectangle(Shape):
13
14     def __init__(self, w, h):
15
16     self.w, self.h = w, h
17
18
19     def area(self):
20
21     return self.w * self.h
22
23
24     def perimeter(self):
25
26     return 2 * (self.w + self.h)
27
28
29     # Shape() # TypeError – cannot instantiate abstract class
30
31     r = Rectangle(4, 5)
32
33     print(r.area()) # 20
34
```

■ You cannot instantiate an abstract class directly. If a subclass does not implement all abstract methods, it too becomes abstract and cannot be instantiated.

13 · Polymorphism

Polymorphism means "many forms" — the same interface (method name or operator) behaves differently depending on the object or context it is applied to.

Operator Polymorphism — the + operator:

Python

```
1 print(2 + 3) # 5 (adds integers)
2 print("Hello" + " World") # Hello World (concatenates strings)
3 print([1,2] + [3,4]) # [1, 2, 3, 4] (extends lists)
```

Method Polymorphism — same method name, different behaviour:

Python

```
1 class Dog:
2     def speak(self): return "Woof!"
3
4 class Cat:
5     def speak(self): return "Meow!"
6
7 class Duck:
8     def speak(self): return "Quack!"
9
10 animals = [Dog(), Cat(), Duck()]
11
12 for animal in animals:
13     print(animal.speak()) # each calls its own version of speak()
14
```

14 · Duck Typing

Duck typing is a Python concept where the **type of an object does not matter** — only whether it has the required attributes or methods. The name comes from the saying: *"If it walks like a duck and quacks like a duck, it's a duck."*

Python

```
1 class Parrot:
2     def speak(self): return "Polly wants a cracker"
3
4 class Robot:
5     def speak(self): return "Beep boop"
6
7 def make_it_speak(thing): # no type check – just calls .speak()
8     print(thing.speak())
9
10 make_it_speak(Parrot()) # works
11
12 make_it_speak(Robot()) # also works – same interface, different type
13
```

■ Duck typing is why Python rarely needs explicit type checks. Focus on what an object can do (its interface), not what it is (its type). This pairs naturally with Polymorphism and makes code more flexible.

Quick Reference — OOP Concepts

Concept	One-liner	Key Syntax
Class	Blueprint for objects	<code>class MyClass:</code>
Object	Instance of a class	<code>obj = MyClass()</code>
<code>__init__</code>	Constructor — runs on object creation	<code>def __init__(self, ...):</code>
<code>@classmethod</code>	Method that receives the class (cls)	<code>@classmethod / def f(cls):</code>
<code>@staticmethod</code>	Method with no self or cls	<code>@staticmethod / def f():</code>
Encapsulation	Wraps data + methods; controls access	<code>self.__private</code>
<code>@property</code>	Pythonic getter/setter	<code>@property / @x.setter</code>
Inheritance	Child gets parent attributes & methods	<code>class Child(Parent):</code>
<code>super()</code>	Call parent method from child	<code>super().__init__(...)</code>
MRO	Method lookup order across inheritance chain	<code>ClassName.__mro__</code>
Abstraction	Force subclasses to implement methods	<code>ABC + @abstractmethod</code>
Polymorphism	Same interface, different behaviour	method override / operator
Duck Typing	Type ignored; only behaviour matters	if it has <code>.speak()</code> → use it